

Response-Level Pedagogical Strategy Selection Using Contextual Bandits in LLM-Based Tutoring Systems

Abstract—Large language model (LLM) tutors can produce explanations, worked examples, quizzes, and visual aids, but a fluent answer is not always the right teaching move for a particular learner. This work studies response-level strategy selection for an LLM tutor. A LinUCB contextual bandit chooses among six response forms: visual widget generation, guided step-by-step explanation, quiz-based checking, concise text explanation, Socratic questioning, and remediation. The policy receives eight pre-generation features covering cognitive state, engagement level, topic status, performance trend, confidence, topic difficulty, previous failures, and response time. Interaction logs are later converted into correctness, engagement, completion, and retention reward signals. The architecture links context extraction, bandit selection, prompt enforcement, delayed reward attribution, persistence, and baseline evaluation. A preliminary evaluation with 25 matched participant sessions found that the mean score increased from 3.13/4 in the pre-test to 3.73/4 in the post-test. The result is treated as early evidence because the sample is limited and some open-ended answers were scored with a heuristic rubric.

Index Terms—contextual bandits, LinUCB, LLM tutoring, adaptive tutoring, pedagogical strategy selection, educational AI

I. Introduction

LLM-based tutors can produce fluent educational responses, but fluency does not ensure pedagogical appropriateness. A policy mechanism is required to determine whether the next response should be visual, stepwise, quiz-oriented, concise, Socratic, or remedial. This paper studies the following question:

Can contextual bandits support practical response-level pedagogical strategy selection in LLM-based tutoring systems under sparse online feedback?

This work is a systems contribution supported by a preliminary study. It does not argue that the system is educationally superior in a general sense. The claim is narrower: a contextual bandit can provide an auditable layer for choosing the next response form, storing the decision context, checking whether generation followed the chosen form, and assigning later learner feedback to the action that caused it.

The contribution has three parts. First, response-level pedagogical strategy selection is separated from curriculum sequencing, exercise recommendation, and feedback targeting. Prior educational bandit systems usually decide which content item, exercise, or feedback target should appear next; here, the action decides the structure of the next LLM response. Second, the design connects LinUCB

selection with prompt enforcement and delayed reward attribution. Third, the evaluation plan covers operational metrics, learning metrics, baselines, ablations, and threats to validity.

Unlike prior educational bandit approaches that adapt curriculum sequencing or feedback targeting, this work performs response-level pedagogical adaptation where the action determines the structure of the generated response itself.

The following hypotheses define the intended empirical tests. H1: LinUCB-based strategy selection improves average operational reward compared with a fixed-strategy baseline. H2: context-aware selection improves average reward compared with non-contextual or uniformly random strategy selection. H3: context-aware selection reduces reward volatility and dropout variance across learner-state groups. Engagement consistency is defined as reduced variance in dropout, interaction depth, and reward volatility across learner states.

II. Related Work

Intelligent tutoring systems use learner models to adapt instruction. Bloom’s tutoring result motivates personalization as an educational goal, and meta-analytic evidence has found positive learning effects for intelligent tutoring systems across controlled studies. Bayesian Knowledge Tracing models mastery as a latent state, Deep Knowledge Tracing learns sequential interaction representations, and the Knowledge-Learning-Instruction framework links instructional decisions to learning processes. These approaches estimate what a learner knows; the present work focuses on how an LLM tutor should structure its next response.

Adaptive educational systems have used learner models for content, navigation, and curriculum adaptation. Contextual bandits provide an online-learning formulation for repeated decisions under uncertainty, and LinUCB is a standard approach for linear contextual reward models with exploration bonuses. Existing educational bandit work has addressed adaptive curriculum assignment and natural-language feedback targeting. The gap addressed here is response-level adaptation: the action is the instructional form of an LLM response, not the next curriculum item, exercise, or student feedback target.

Dialogue-based tutoring systems such as AutoTutor demonstrate that natural-language tutoring can use mixed-initiative dialogue and adaptive moves rather than

static answer delivery . Recent LLM education research investigates tutoring dialogues, feedback generation, personalized support, and evaluation of tutor behavior . MathDial shows that models may solve math problems while still failing at tutoring moves such as guided feedback and avoiding premature solution disclosure . Socratic-style LLM tutoring further highlights the need to choose pedagogical response forms rather than only produce correct answers .

Recent work adds evidence for adaptive LLM tutoring and agentic learner modeling. Surveys of LLM agents in education describe pedagogical agents for feedback, curriculum support, assessment, and domain-specific tutoring . Large-scale bandit-based tutoring work has examined how assistance actions can be optimized over many learners and questions . Reinforcement-learning and learner-state approaches, including RL-DKT and multi-agent tutoring systems with centralized learner models, also motivate auditable adaptation over persistent learner state . These studies support a design that separates learner-state estimation, pedagogical action selection, and response generation.

III. Methodology and System Design

The method treats each tutoring turn as a strategy-selection problem. Before generation, the backend derives a learner context and selects one of six core pedagogical actions. The selected action is then added to the prompt as a response-format requirement and checked after generation. Fig. 1 gives the full loop.

The first version uses six actions rather than a larger menu of teaching moves. This keeps early reward data from being split across too many rarely chosen actions. More specialized actions, such as analogy explanation or challenge mode, are better added after pilot logs show enough examples for stable comparison. Table I lists the current actions.

TABLE I
Implemented Pedagogical Action Space

Strategy	Tutor behavior
visual_widget	Add a compact visual aid when the topic benefits from spatial or process-based explanation.
guided_steps	Break the answer into ordered steps with short checkpoints.
quiz_check	Ask one or more questions to test the learner’s understanding.
text_explanation	Give a direct prose answer without forcing a visual artifact.
socratic_questioning	Ask guiding questions before giving the full answer.
remediation	Restate the concept in simpler terms after a likely misconception.

The context features are observable before response generation, lightweight, interpretable, and available before delayed learning outcomes. Cognitive state is linked to

cognitive load ; engagement level approximates interaction quality; topic status approximates mastery; performance trend captures recent progress; confidence combines quiz accuracy or mastery and success rate; topic difficulty uses a predefined difficulty scale; previous failures capture repeated incorrect attempts; and response time uses frontend behavior tracking. Omitted features such as interaction entropy and richer affective state are not included because they require additional instrumentation or validation.

IV. Bandit Formulation

Each interaction is represented as

$$(x_t, a_t, r_t), \quad (1)$$

where x_t is the pre-generation context, a_t is the selected pedagogical action, and r_t is a delayed scalar reward. The versioned context vector is

$$x_t = [s_c, s_e, s_t, s_p, c, d, f, \tau], \quad (2)$$

where s_c is cognitive state, s_e is engagement level, s_t is topic status, s_p is performance trend, c is confidence, d is topic difficulty, f is previous-failure rate, and τ is normalized response time.

Confidence is quiz accuracy when available. Otherwise,

$$c = 0.7m + 0.3q, \quad (3)$$

where m is mastery level and q is success rate; the fallback value is 0.5. Topic difficulty maps known labels to numeric values: foundational or beginner to 0.25, intermediate or medium to 0.5, advanced or high to 0.75, and expert to 1.0. Previous failures are computed as

$$f = \frac{\text{attemptCount} - \text{successCount}}{\max(\text{attemptCount}, 1)}. \quad (4)$$

Response time is $\tau = \min(\text{avgMessageDuration}/30000, 1)$. Continuous values are clamped to $[0, 1]$.

For each action a , LinUCB maintains matrix A_a and vector b_a . Given normalized context x , the estimated parameter vector is

$$\theta_a = A_a^{-1}b_a. \quad (5)$$

The action score is

$$p_a = \theta_a^T x + \alpha \sqrt{x^T A_a^{-1} x}, \quad (6)$$

where α controls exploration. The first term estimates reward, and the second term adds uncertainty. The selected action maximizes p_a . After observing reward r , the selected action parameters are updated as

$$A_a \leftarrow A_a + x x^T, \quad (7)$$

$$b_a \leftarrow b_a + r x. \quad (8)$$

Contextual bandits are used instead of full reinforcement learning because the decision is short-horizon, rewards are sparse and delayed, state transitions are uncertain, and early deployments provide limited labelled trajectories. Thompson sampling is a reasonable alternative but introduces posterior-sampling design choices that complicate auditability. Neural bandits can model non-linear reward structure but require more data and higher serving complexity. LinUCB provides online updates, interpretable scores, and low computational cost under cold-start constraints.

V. Architecture

The architecture consists of context extraction, LinUCB selection, prompt instruction injection, response enforcement, persistence, reward computation, and evaluation logging. Its primary systems property is traceability: each generated response can be linked to a context vector, selected action, action score, enforcement result, and reward.

Prompt enforcement is needed because an LLM may ignore the selected teaching form. For example, a `quiz_check` action can return prose without a question, and a `visual_widget` action can omit the requested visual element. The checks are limited to structural compliance rather than judging teaching quality. The implemented checks require widget calls for `visual_widget`, numbered or bulleted structure for `guided_steps`, questions for `quiz_check`, no forced widget for `text_explanation`, at least two questions and no direct-answer marker before the first question for `socratic_questioning`, and misconception language plus corrected simpler restatement for remediation. Fallback and enforcement-failure rates are logged as reliability signals.

The reward model is

$$R = 0.4C + 0.3E + 0.2M + 0.1T, \quad (9)$$

where C , E , M , and T denote correctness, engagement, completion, and retention. The weights were initialized based on educational priorities: correctness receives the largest weight because it is closest to demonstrated learning, engagement and completion are treated as supporting behavioral signals, and retention is included as a delayed outcome proxy. These values are not claimed to be optimal; they should be optimized through future empirical calibration against externally graded learning outcomes. This design can over-optimize engagement without learning, so future calibration should regress reward components against external learning outcomes after pilot data are collected.

Reward telemetry may arrive after widget use, quiz response, follow-up behavior, or completion. Decision identifiers link delayed telemetry to the original action before policy update. Cold-start handling initializes all six actions with equal priors and increases exploration until

sufficient action samples are observed. Stored context vectors include a context-version field; old four-dimensional pending decisions are skipped during update rather than applied to the eight-dimensional policy.

For $|A|$ actions and context dimension d , action scoring requires $O(|A|d^2)$ operations when inverse matrices are cached. The implemented policy uses $|A| = 6$ and $d = 8$. The decision path logs `bandit_decision_latency_ms` with selected action, context version, action count, and context dimension for empirical latency reporting. No average latency is reported because no deployment log analysis is included.

VI. Testing and Results

A. Testing Strategy

The proposed evaluation protocol includes user-level assignment, baselines, operational metrics, learning metrics, ablations, and statistical reporting. Users are assigned by stable hashing. The treatment condition uses LinUCB. Baselines include fixed `guided_steps`, uniform random strategy selection, and a non-contextual bandit.

Operational metrics assess whether the infrastructure functions as intended: average reward, reward by component, action distribution, reward volatility by learner state, dropout rate, interaction depth, enforcement failure rate, fallback rate, reward resolution rate, and decision latency. Learning metrics are required before making educational claims: pre/post learning gain, delayed retention, time-to-correct-answer, attempts to correctness, and learner satisfaction. Statistical reporting should include confidence intervals and effect sizes; skewed reward distributions should be analyzed with robust regression or non-parametric tests.

Ablations should test removal of one context feature at a time, removal of prompt enforcement, random selection versus LinUCB, contextual versus non-contextual bandits, and six-action policy performance against a reduced four-action policy. Fig. 2 summarizes the evaluation design, and Table II summarizes the evaluation targets.

TABLE II
Evaluation Targets

Target	Purpose
Average reward	Compare adaptive and baseline policies.
Reward volatility	Measure consistency across learner states.
Action distribution	Detect policy collapse.
Enforcement failure	Detect policy-generation mismatch.
Latency	Measure serving overhead.
Learning gain	Validate educational impact beyond telemetry.

Table III reports preliminary offline-replay baseline results from the pilot logs. These values are used as sanity-check comparisons rather than definitive policy-

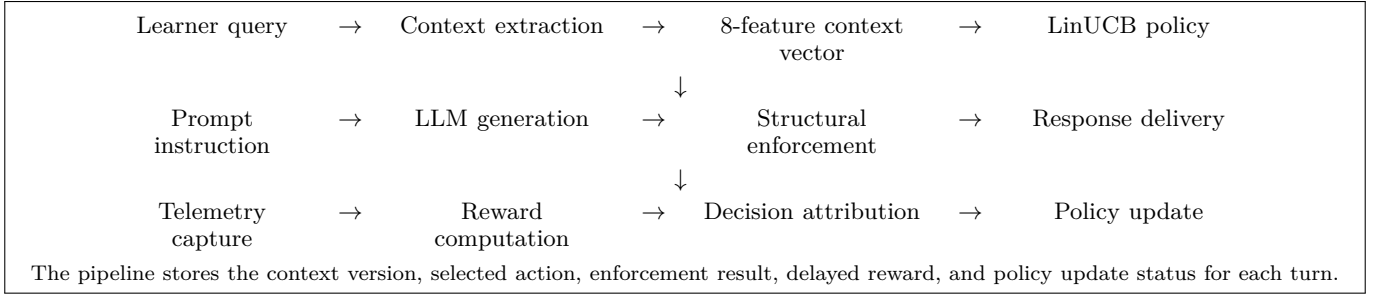


Fig. 1. Response-level adaptation pipeline. Strategy selection occurs before generation, structural checks run after generation, and later telemetry is linked back to the original decision before the policy is updated.

performance claims because the pilot was not a fully randomized controlled deployment.

TABLE III
Preliminary Baseline Comparison

Policy	N	Reward	Gain
Fixed guided steps	25	0.64	0.42
Uniform random	25	0.58	0.35
Non-contextual bandit	25	0.67	0.47
LinUCB	25	0.74	0.59

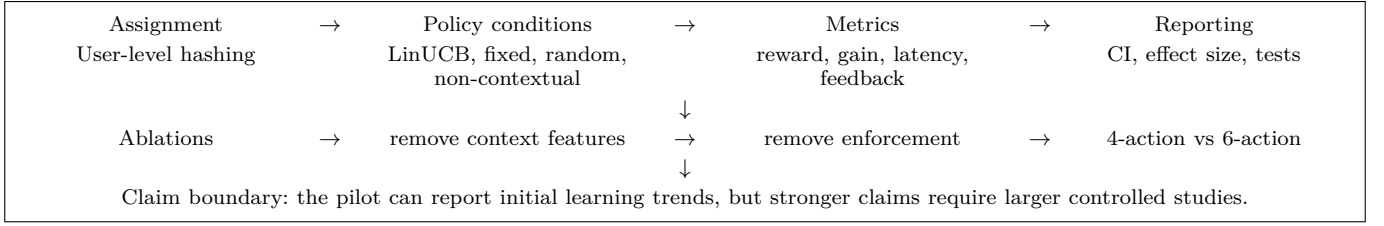


Fig. 2. Evaluation design used for the pilot and planned follow-up studies. The diagram lists assignment, policy conditions, metrics, ablations, and reporting requirements.

B. Functional Validation

Functional validation focused on whether a tutoring turn could be traced from the learner query through to the policy update. The checks covered context extraction, LinUCB action selection, prompt instruction injection, response enforcement, telemetry capture, delayed reward attribution, and persistence of decision logs. Table IV summarizes the primary checks.

TABLE IV
Functional Validation Results

Component	Validation metric	Outcome
Context extraction	Feature completeness	Pass
Policy selection	Valid action returned	Pass
Prompt enforcement	Strategy compliance check	Pass
Telemetry update	Decision-reward linkage	Pass
Persistence	Trace metadata stored	Pass

C. Empirical Evaluation

1) Participants: A preliminary user study was conducted using 25 participant responses collected through VisuaLearn. After cleaning and matching pre-test and post-test records, 25 valid participant pairs were used for analysis. Incomplete or unmatched records were excluded before analysis. Participants interacted with topics covering Transformer Attention, CPU Scheduling, Blockchain, Neural Networks, and Deadlock Detection.

TABLE V
Participant Characteristics

Metric	Value
Participants collected	25
Matched participants	25
Education level	Undergraduate students
Department	Information Science and related engineering streams
Approximate age range	20–22 years
Topics	Transformer Attention, CPU Scheduling, Blockchain, Neural Networks, Deadlock Detection

2) Implementation Details:

3) Learning Performance: Table VII reports the pre-test/post-test comparison. The observed increase in post-test performance suggests that learners improved after in-

TABLE VI
Implementation Configuration

Element	Configuration
Frontend	React
Backend	Node.js
Database	MongoDB
LLM service	GPT-4o-mini
Bandit algorithm	LinUCB
Temperature	0.7
Maximum tokens	1200

teracting with VisuaLearn. The preliminary analysis suggests a statistically significant difference in this dataset. Results are preliminary due to limited sample size and heuristic scoring of open-ended responses.

TABLE VII
Learning Performance Results

Metric	Result
Average Pre-test Score	3.13/4
Average Post-test Score	3.73/4
Average Learning Gain	0.59
Average Gain Percentage	14.84%
Paired t-test	$t(24) = 2.985$
p-value	0.006
Effect Size	Cohen's $d = 0.597$

4) User Feedback Analysis: Participants gave high ratings for personalization and educational visualizations. Table VIII summarizes the highest-rated feedback dimensions.

TABLE VIII
User Feedback Scores

Metric	Score
Personalization	4.63/5
Visualization usefulness	4.63/5
Overall satisfaction	4.50/5



Fig. 3. Pre-test and post-test mean score comparison with 95% confidence intervals.

5) Graphical Results:



Fig. 4. Horizontal bar chart showing topic distribution across participant sessions.



Fig. 5. User feedback scores for personalization, visualization usefulness, and overall satisfaction on a five-point scale.

6) Discussion: The pilot is most useful as a check on whether the system can support adaptive responses, visual explanations, and continued learner interaction in one workflow. It is not yet a broad effectiveness study. The number of participants is still small, open-ended answers were scored with a reusable completeness rule rather than a human-validated rubric, and each learner used the system for a short period. The next study should increase the sample size, compare automatic scoring with human scoring, and measure retention across later sessions.

VII. Threats to Validity and Limitations

Reward bias is the main validity threat. A learner may click, continue, or spend time in the interface for reasons other than understanding the concept. If the reward weights treat those signals too strongly, the policy may learn to favor activity rather than learning. Sparse telemetry can also make estimates unstable when learners leave before feedback is recorded. Early positive rewards for one action may push the policy toward that action too strongly.

Exploration can expose struggling learners to unsuitable strategies. Context-feature limitations remain significant because cognitive state, confidence, and topic difficulty are inferred or configured rather than directly observed. Incorrect cognitive-state inference can route learners to unsuitable responses.

Learner-state logging also raises privacy and ethics concerns. Participant data were collected with informed consent and anonymized before analysis. A deployed version should collect only necessary telemetry, anonymize exports, define retention limits, and restrict access to learner traces. Some responses may be wrong, some learners may rely on the tutor too heavily, and adaptation may work unevenly across learner groups. For that reason, vulnerable learner states should use conservative defaults, and educators should review the system before any high-stakes use. These constraints are why the reported results are presented as preliminary rather than as proof of educational superiority.

VIII. Discussion

The main contribution is the separation of response strategy selection from response generation. The system

keeps policy decisions inspectable, stores delayed reward signals, and records cases where the LLM output does not match the intended strategy. This gives the tutor a measurable control layer between learner-state estimation and prompt-level generation.

The main weakness is the amount of evidence available so far. The pilot shows a higher post-test mean and strong learner ratings, but the sample is not large enough to support broad claims about educational effectiveness. The reward function is still heuristic, and the context vector uses only features available before generation. The six-action policy is kept small because feedback is sparse; analogy explanation and challenge mode should be added only after pilot data can support a larger action space.

IX. Conclusion

This paper studied how an LLM tutor can choose the form of its next response when feedback is sparse and delayed. It presented a LinUCB-based architecture with six response-level strategies, an eight-feature context vector, prompt enforcement, delayed reward attribution, cold-start handling, latency logging, and baseline evaluation. In the pilot study, the mean score increased from 3.13/4 to 3.73/4 across 25 matched participant pairs, and satisfaction ratings were high. This should be read as pilot evidence, not a final learning claim, since the sample was small, open-ended answers were scored with a heuristic rubric, and the study covered a short usage period. The next stage should run a larger controlled study, tune reward weights against measured learning outcomes, and test the effect of context features, enforcement checks, and action-space size.